SUDOKU GAME DOCUMENTATION

4×4 Sudoku Game | Java Swing Desktop Application

Table of Contents

Section	Topic
1	Introduction
2	Project Objectives
3	Scope and System Requirements
4	System Architecture and Design
5	Development Progress: Week 1–10
6	Algorithm and Game Logic
7	Flowchart
8	Implementation and Game Output
9	Testing, Results and Discussion
10	Conclusion and Future Improvements

Project Summary

This project is a simple 4×4 Sudoku desktop game developed in Java using the Swing GUI library. The player fills empty cells with numbers 1–4 while keeping the original clues unchanged.

1. Introduction

Sudoku is a logic-based number puzzle in which the player fills empty cells while following specific rules. This project develops a beginner-friendly 4×4 Sudoku game using Java and the Swing graphical user interface (GUI) library. A smaller grid makes the game easier to learn and demonstrate.

The application displays fixed numbers and empty cells. The player enters numbers in available cells and uses the game controls. Through this project, Java classes, arrays, loops, conditions, GUI components and event handling can be practiced.

1.1 Purpose of the Project

The purpose is to build a simple interactive game and understand how program data, the graphical interface and user actions work together. The project also provides practice in planning, coding, testing, debugging and technical documentation.

1.2 Basic Rules of 4×4 Sudoku

- Use the numbers 1, 2, 3 and 4.
- Each row must contain each number only once.
- Each column must contain each number only once.
- Each 2×2 sub-grid must contain each number only once.
- Original clue numbers should not be changed.

1.3 Project Overview

Puzzle layouts are stored in two-dimensional integer arrays. A value of 0 represents an empty cell. Non-zero values are displayed as clues, while empty cells are available for player input. Swing components such as JFrame, JPanel, JTextField and JButton are used to build the interface.

2. Project Objectives

1. Develop a clear 4×4 Sudoku board with easy-to-understand controls.
2. Apply Java classes, arrays, loops, conditions and methods.
3. Display puzzle clues in their correct positions.
4. Protect fixed clues and allow input in empty cells only.
5. Provide controls for starting a new puzzle and clearing player entries, where implemented.
6. Restrict entries to the numbers 1–4 and inform the player about invalid input, where implemented.
7. Use a two-dimensional array to store and manage puzzle data.
8. Test the board, editable cells and control buttons.
9. Prepare clear documentation for academic submission.

Expected Outcome

The expected outcome is a small interactive desktop game with a readable grid, stored puzzles and basic controls. Further improvements can include complete rule checking, hints, a timer, scoring and additional puzzles.

3. Scope and System Requirements

3.1 Project Scope

The project focuses on a compact 4×4 Sudoku puzzle for learning and classroom demonstration. It includes a puzzle grid, fixed clues, editable empty cells and basic controls. Online play, user accounts and automatic puzzle generation are outside the basic scope.

3.2 Functional Requirements

- Open a game window and display a 4×4 grid.
- Load a stored puzzle layout.
- Keep original clue cells protected.
- Allow the player to enter values in empty cells.
- Provide the available game controls, such as Clear and Check.
- Validate entries and check completion if those features are implemented.

3.3 Non-functional Requirements

- Usability: the interface should be simple and readable.
- Reliability: ordinary user actions should not crash the program.
- Maintainability: puzzle data and event-handling code should be organized.
- Portability: the program should run where a compatible Java environment is installed.
- Performance: normal actions should respond quickly.

3.4 Hardware and Software Requirements

Category	**Requirement**
Hardware	Computer or laptop and keyboard
Operating system	Windows, Linux or another supported system
Programming language	Java
IDE	Apache NetBeans IDE
GUI library	Java Swing

4. System Architecture and Design

The application follows an event-driven design. The user interacts with the interface; Swing generates an event, and the related event handler processes the action. The puzzle array stores the original clues, while text fields display the board.

4.1 Main Components

10. JFrame: creates the main game window.
11. JPanel: holds and arranges the 16 cells.
12. JTextField: displays a clue or accepts input in an empty cell.
13. Integer arrays: store puzzle layouts; 0 means an empty cell.
14. JButton: provides controls for available game actions, such as Clear and Check.
15. Event handlers: respond to button actions and process user input.

4.2 Data Representation

The board is represented by a two-dimensional array with four rows and four columns. For example, {1, 0, 3, 0} means that 1 and 3 are fixed clues, while the second and fourth cells are empty.

4.3 User Interaction

When a puzzle is loaded, the program reads its array and displays all 16 values. Fixed clue cells are made non-editable and empty cells are left available for input. When a new puzzle is selected, the board should refresh so that values from the previous game do not remain.

5. Development Progress: Week 1–10

The project was organized into ten stages. Each week focused on a specific part of planning, development or submission. The descriptions below explain the activity, how it was approached and its purpose.

Week 1 — Topic Selection and Planning

The project topic was selected as a 4×4 Sudoku Game. The basic idea, target users and expected outcome were identified. The initial plan was to create a small desktop game that could be explained and demonstrated in class.

Outcome: A clear project direction and an initial feature list.

Week 2 — Requirements and Game Rules

The main requirements were listed: display a 4×4 board, use numbers 1–4, preserve the given clues and allow the player to fill empty cells. The rules for rows, columns and 2×2 sub-grids were reviewed.

Outcome: A basic requirements list and an understanding of the game rules.

Week 3 — Interface Planning

The layout of the main window was planned, including the grid, cell size, title and control buttons. The goal was to keep the interface readable and simple for a first-time player.

Outcome: A planned layout for the game window and its controls.

Week 4 — Basic Game Interface Development

The Java project was set up in NetBeans. The main Swing window was created using JFrame, and panels/layouts were used to organize the game components.

Outcome: A basic desktop window in which the Sudoku interface could be built.

Week 5 — 4×4 Grid Implementation

A two-dimensional array of text fields was used to represent the 4×4 board. The cells were arranged in four rows and four columns. Alignment, font size and spacing were adjusted to make the grid easy to read.

Outcome: A visible 4×4 Sudoku grid containing 16 cells.

Week 6 — Game Control Logic

The logic for responding to user actions was developed. Button events were connected to their respective operations, such as clearing entries or loading another stored puzzle, according to the program implementation.

Outcome: Controls that can process user actions when correctly connected.

Week 7 — Puzzle Setup and Fixed Clues

Puzzle layouts were stored in integer arrays. Non-zero values represented given clues and zero values represented empty cells. The selected puzzle data was displayed in the grid.

Outcome: Stored puzzle layouts with clues shown in the correct positions.

Week 8 — Input and User Features

Input handling was added to the editable cells. The design allows the player to enter numbers in empty cells while fixed clues remain protected. Input restrictions and user messages can be included to make the game easier to use.

Outcome: A more interactive board for entering answers and using game controls.

Week 9 — Debugging and Testing

The program was checked for interface and control problems. The grid display, clue protection, editable cells and available controls were reviewed. Problems found during testing were corrected where possible, and the program was run again.

Outcome: A more stable program and a list of features that still need verification.

Week 10 — Documentation and Final Submission

The project structure, objectives, requirements, weekly progress, algorithm, flowchart, output and testing details were organized into this report. The final program should be run once more before submission so that screenshots and test results match the actual version.

Outcome: A clear project report and organized files for submission to the teacher.

5.1 Challenges and Solutions

One challenge is separating fixed clues from values entered by the player. The original puzzle array can be used to identify clue cells and disable editing for those cells. Another challenge is refreshing the whole board when starting a new puzzle. Reloading all 16 cells from the selected puzzle helps prevent old entries from remaining on the screen. Each button should be tested in the final running version.

6. Algorithm and Game Logic

6.1 Main Algorithm

16. Start the program.
17. Create the game window and grid.
18. Select a puzzle from the stored puzzle arrays.
19. Read every cell in the selected puzzle.
20. Display non-zero values as fixed clues and leave zero-valued cells editable.
21. Wait for the player to enter a value or select a control.
22. Validate the input or process the selected button action.
23. If full checking is implemented, check the rows, columns and 2×2 sub-grids.
24. Show a success message when the puzzle is correctly completed.
25. Continue playing, start another puzzle or close the application.

6.2 Pseudocode

BEGIN

```
Create game window
Load selected puzzle
FOR each row and column
   IF cell value is not 0
      Display clue and disable editing
   ELSE
      Display empty editable cell
   END IF
END FOR
WHILE game is open
   Wait for player action
   Validate input or process button action
   Refresh board if required
END WHILE
END
```

6.3 Input Validation

The intended input is a single digit from 1 to 4 in an empty cell. Other characters or values outside this range should be rejected or reported if validation is implemented. A complete Sudoku checker must also verify each row, column and 2×2 sub-grid.

7. Flowchart

Main sequence of the 4×4 Sudoku Game:

START

↓

Create game window and initialize components

↓

Load selected puzzle from puzzle array

↓

Display fixed clues and editable empty cells

↓

Wait for player input or button click

↓

Is the input a number from 1 to 4?

├─ NO → Show warning and request valid input

└─ YES → Place number in selected cell

↓

Is the puzzle completed correctly? (if checking is implemented)

├─ NO → Continue playing

└─ YES → Show winning message

↓

Choose Clear, Check or Close

↓

END / RESTART

Note: Clear removes player entries where supported. Close ends the application. The exact validation and winning-message steps depend on the features available in the submitted code.

8. Implementation and Game Output

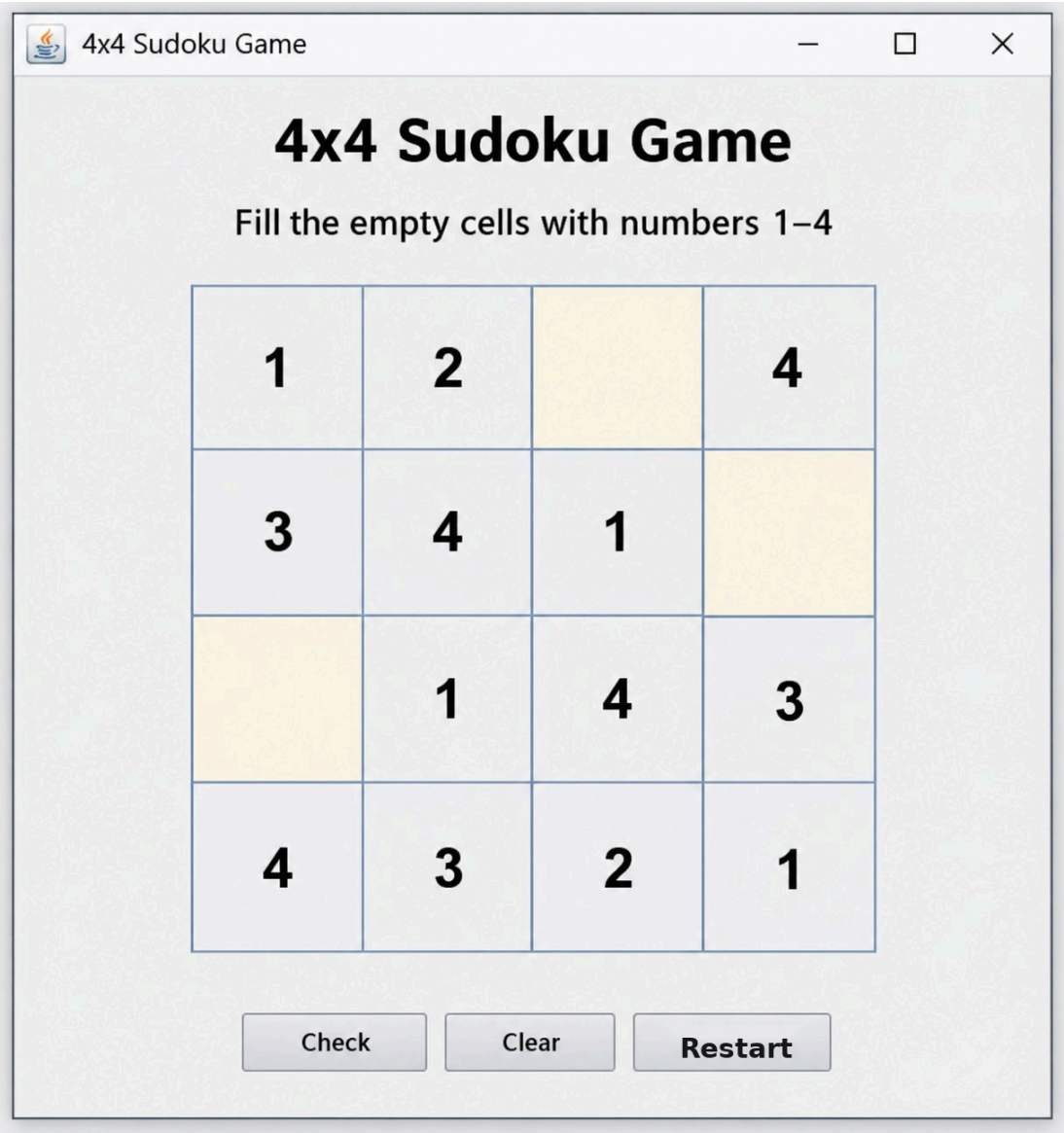
The program is written in Java and uses Swing for the graphical interface. The main class can extend JFrame. A two-dimensional array of JTextField objects represents the cells, while JPanel and GridLayout arrange the cells in a 4×4 board. Buttons are connected to event handlers to process game actions.

8.1 Main Implementation Points

- Create the main window and set its title and size.
- Build a 4×4 panel and add 16 text fields.
- Load the selected puzzle array into the fields.
- Disable editing in cells that contain fixed clues.
- Connect buttons to their intended actions.
- Run the application and verify the actual output before taking a screenshot.

8.2 Game Output

The screenshot below shows a clear 4×4 Sudoku game interface with the puzzle grid and available controls.



9. Testing, Results and Discussion

The following test cases should be checked on the final running program. Mark a test as passed only after confirming the action works in the submitted version.

No.	Test action	Expected result
1	Run the application	The game window opens and shows a 4×4 grid.
2	Load a puzzle	Fixed clues appear in the correct cells.
3	Try to edit a clue cell	The original clue cannot be changed.

4	Enter a number from 1 to 4	The value is accepted in an editable cell.
5	Enter invalid input	Input is rejected or a warning appears, if validation is implemented.
7	Press Clear	Player entries clear while fixed clues remain, if implemented.
8	Complete the puzzle	A success message appears if full checking is implemented.

9.1 Results and Discussion

The 4×4 board provides a simple way to practice array indexing and GUI event handling. The final result should be evaluated by running the submitted program and checking every available control. Features that are not present or have not been tested should not be reported as successful.

10. Conclusion and Future Improvements

This project demonstrates how Java and Swing can be used to create a small interactive desktop game. It combines puzzle data, a grid-based interface, editable cells and event-driven controls. The ten-week development plan covers topic selection, requirements, interface planning, implementation, testing and documentation.

Future improvements may include complete row/column/sub-grid validation, hints, a score counter, a timer, difficulty levels, more stored puzzles and improved keyboard navigation. These features can make the game more useful and engaging.

Final Submission Checklist

- Run the Java program in NetBeans.
- Check that the available controls work as intended.
- Insert a real screenshot of the final game output.
- Review spelling, page numbers and table alignment.
- Submit the Java source code together with this documentation.